

DRY-RUN GUIDE | gym equipments2.py
Source: ca-2\psup\gym equipments2.py

CORE BEHAVIOR

Stores equipment counts in a dictionary, applies a booking increment, adds/removes entries safely, queries usage, then prints total and busiest remaining equipment.

REPRESENTATIVE INPUT

Initial treadmill=10, bike=5, rower=8; book treadmill +2; add mats=3; remove bike; query bike.

STEP TRACE

treadmill=12; add mats=3; pop bike=5; lookup removed bike defaults to 0; remaining total=12+8+3=23; treadmill is busiest.

EXPECTED OUTPUT / RETURN

Progress messages; final 'Total users: 23' and 'Busiest equipment: treadmill (12)'.

ACTUAL SOURCE EXCERPT

Verbatim excerpt from ca-2\psup\gym equipments2.py; line numbers refer to the original file. No source was modified.

```
001 | # Gym Equipment Tracker
002 | name1 = input().strip()
003 | users1 = int(input())
004 | name2 = input().strip()
005 | users2 = int(input())
006 | name3 = input().strip()
007 | users3 = int(input())
008 | di = {name1: users1, name2: users2, name3: users3}
009 |
010 | print(f"{name1} : {di[name1]} users")
011 | print(f"{name2} : {di[name2]} users")
012 | print(f"{name3} : {di[name3]} users")
013 | name4 = input().strip()
014 | users4 = int(input().strip())
015 | if name4 in di:
016 |     di[name4] += users4
017 | print(f"Updated {name4} : {di[name4]} users")
018 | name5 = input().strip()
019 | users5 = int(input())
020 | di[name5]=users5
021 | print(f"Added {name5} : {di[name5]} users")
022 | name6 = input().strip()
023 | if name6 in di:
024 |     print(f"Removed {name6} with {di[name6]} users")
025 |     del di[name6]
026 | name7 = input().strip()
027 | if name7 in di:
028 |     print(f"Usage of {name7}: {di[name7]} users")
029 | total_users = sum(di.values())
030 | if di:
031 |     busiest_name, busiest_users = max(di.items(), key=lambda item: item[1])
032 | else:
033 |     busiest_name, busiest_users = "No equipment", 0
034 | print(f"Total users: {total_users}")
```

```
035 | print(f"Busiest equipment: {busiest_name} ({busiest_users})")
```

DRY-RUN NOTE

Values are representative. For function-only modules the guide uses an illustrative direct call and explains its return; quirks/errors describe actual behavior.